

Neuronale Netze & Künstliche Intelligenz





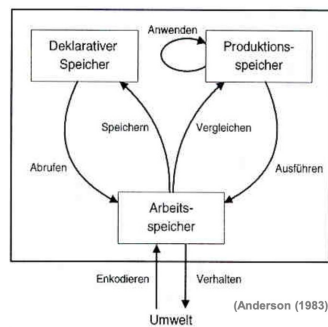
Gliederung

- ACT-R
- Maschinelles Lernen
- Neuronale Netze

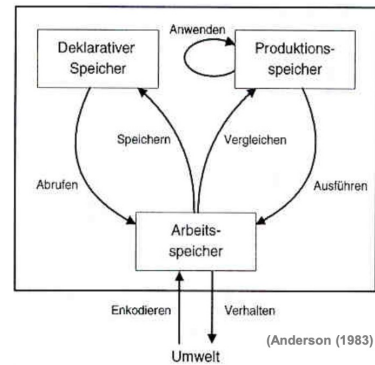
- „Unified Theory of Cognition“ als globales Ziel der Kognitionswissenschaft
 - **Kognitive Architekturen** stellen einen Ansatz zur Entwicklung dieser Theorien dar
 - Definition **allgemeiner Strukturen und Mechanismen**, die für eine Klasse von Repräsentationssystemen gelten
 - **Kognitive Architektur:**
 - Strukturen; Speicher (leer)
 - Prozesse (allgemein) übernimmt
-
- **Kognitives Modell:**
 - Spezifische (Wissens-)Inhalte
 - Spezifische Prozesse

- Ein Repräsentationssystem, das das Wissen über einen spezifischen Gegenstandsbereich bzw. über eine Aufgabe modelliert, setzt eine generelle Architektur voraus, die u.a. beschreibt:
 - Welche Wissensarten hierfür relevant sind
 - Wie Wissen gespeichert und abgerufen wird
 - Wie Wissen benutzt wird, um Probleme zu lösen
 - Wie Wissen erworben wird
 - Wie sich jemand vom Laien zum Experten entwickelt
- Eine kognitive Architektur ist eine computationale Plattform, in der kognitive Modelle „programmiert“ werden können, die menschliches Verhalten simulieren und damit vorhersagen bzw. erklären
- Eine kognitive Architektur sollte nur Modelle zulassen, die kognitions- und neuropsychologisch plausibel sind

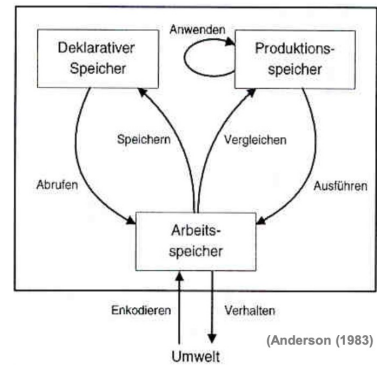
- Eine kognitive Architektur besteht aus allgemeinen Strukturen (z.B. „Speicher“) und Prozessen, die auf diesen Strukturen operieren (z.B. „etwas aus dem Speicher abrufen“)
- **ACT-R** ist eine bekannte kognitive Architektur
 - „Adaptive Control of Thought – Rational“
 - Von Anderson (1974) als ACT entwickelt, in 1993 Weiterentwicklung zu ACT-R



- **Deklarativer Speicher** für Faktenwissen
- **Produktionsspeicher** für Handlungswissen
- **Arbeitsspeicher** als eigenständiger Speicher dargestellt, aber als aktuell aktivierter Teil des deklarativen Wissens konzipiert
- Der Arbeitsspeicher ist der Ort der **bewussten Informationsverarbeitung** und stellt die Verbindung zur Außenwelt her



- **Enkodieren:**
 - Informationen, die über Sinne aufgenommen werden, werden in einem mentalen Code übersetzt und im Arbeitsspeicher weiterverarbeitet
- **Verhalten:**
 - Informationen im Arbeitsgedächtnis können Aktionen auslösen, die nach außen gerichtet sind, und zu beobachtbarem Verhalten führen
- **Speichern:**
 - Informationen vom Arbeitsspeicher werden dauerhaft in die deklarative Wissensstruktur integriert
- **Abruf:**
 - Aktuelle Informationen werden vom deklarativen Wissen in den Arbeitsspeicher überführt



▪ Vergleichen

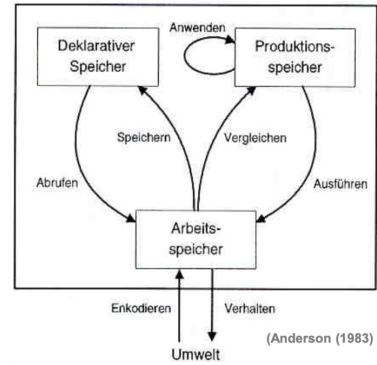
- Produktionen bestehen aus einem WENN und einem DANN Teil
- Beim Vergleichen wird geprüft, ob der Bedingungsteil mit aktiven Propositionen im Arbeitsgedächtnis übereinstimmt

▪ Anwenden:

- Die Produktion wird auf die Proposition angewandt (Der DANN Teil wird „generiert“)

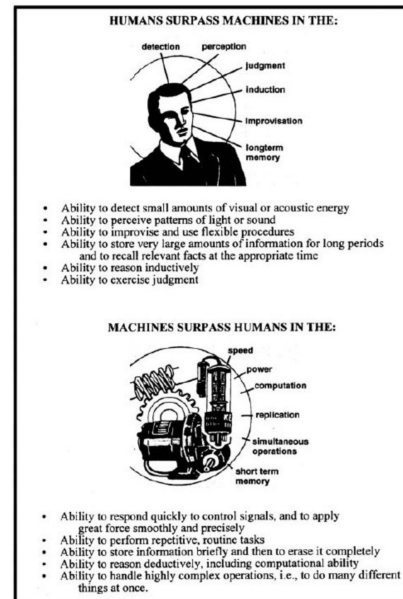
▪ Ausführen:

- Die Produktion wird ausgeführt und neue Inhalte (DANN Teil) gelangt in den Arbeitsspeicher



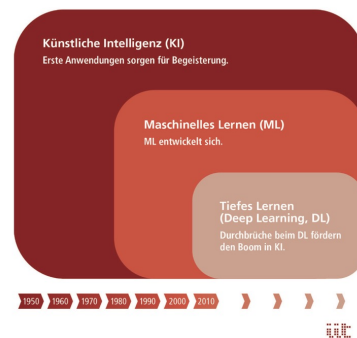
- Kognitive Architekturen bieten Modelle, mit denen Kognition simuliert und vorhergesagt werden kann
- Eine bekannte Architektur ist ACT-R
- ACT besteht im Wesentlichen aus drei Speichern:
 - Deklarativer Speicher
 - Produktionsspeicher
 - Arbeitsspeicher

- HABA-MABA Listen
 - Teilung zwischen Fertigkeiten von Menschen und denen von Maschinen
 - Automation von Tätigkeiten basierte auf HABA-MABA Listen
 - Listen, die aufzeigen, was Menschen und Maschinen jeweils besser können
 - Basiert auf strikter Aufgabenteilung zwischen Mensch und Maschine ohne Kollaboration



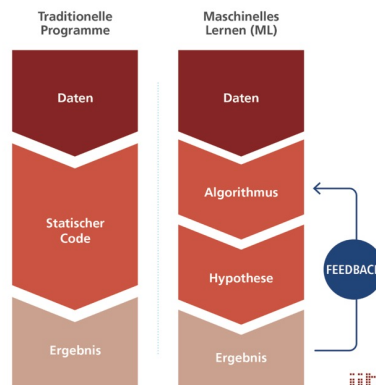
Bradshaw et al. (2011)

- Definition KI
 - „Teilgebiet der Informatik, das sich mit der Automatisierung von intelligentem Verhalten befasst“
 - Eine genaue Definition ist schwer, aber die meisten Definitionen haben gemeinsam, dass ein System entwickelt werden soll, das eigenständig komplexe Probleme bearbeiten kann
 - Anfangs ging es bei KI um Spiele und mathematische Repräsentationssysteme von Wissen und Entscheidungen
 - Später stehen maschinelles Lernen und seit kurzem Deep Learning im Mittelpunkt



Kirste & Schürholz (2019)

- Maschinelles Lernen
 - „Wie bringt man ein Computerprogramm, das eine bestimmte Aufgabe hat, dazu, aus Erfahrungen zu lernen und mit diesen Erfahrungen die Aufgabe in Zukunft besser zu erfüllen?“ ([Kirste & Schürholz, 2019](#))
 - Entscheidungsregeln passen sich über eine Rückkopplung an das Erlernete an



[Kirste & Schürholz \(2019\)](#)

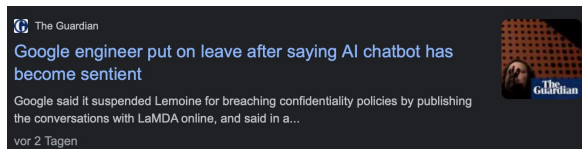
- Maschinelles Lernen
 - Unterscheidung zwischen (a)
 - **Überwachtes** Lernen (Supervised Learning)
 - Programm bekommt Daten und eine Interpretation / ein Ergebnis
 - Ziel: Regeln zu entdecken, um aus neuen Daten passende Ergebnisse zu generieren
 - Beispiel: Apfel in Bildern erkennen
 - **Unüberwachtes** Lernen (Unsupervised Learning)
 - Programm bekommt Daten, aber keine Interpretation / kein Ergebnis
 - Ziel: Ähnlichkeiten in Daten erkennen und bspw. Cluster bilden
 - Beispiel: Empfehlungen für ähnliche Serien bei Netflix
 - **Verstärktes** Lernen (Reinforcement Learning)
 - Programm interagiert mit Umwelt und lernt direkt aus Erfahrungen (z.B. Belohnung wie Gewinn eines Spiels)
 - Ziel: Konsequenzen einer Handlung merken und Belohnung maximieren
 - Beispiel: Autonome Fahrzeuge, Anwendung in Robotern

- Maschinelles Lernen
 - Unterscheidung zwischen (b)
 - **Offline**
 - Das Lernen findet getrennt vom Anwendungsszenario statt
 - Das Gelernte wird dann angewandt, aber nicht mehr verändert
 - **Online**
 - Systeme lernen innerhalb des Anwendungsszenarios
 - Das Gelernte wird ständig angepasst

- 3 Phasen bei dem Einsatz künstlicher Intelligenz
 - **Trainingsphase**
 - Das Modell wird anhand von Trainingsdaten trainiert
 - **Validierungsphase**
 - Das beste Modell wird ausgewählt bzw. Parameter werden angepasst
 - **Testphase**
 - Das Modell wird anhand von neuen, nicht bereits gesehen Daten (Testdaten) getestet
 - Zentrale Frage: Funktioniert das Modell hier genau so gut?

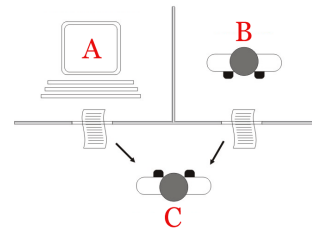


- Unterscheidung von 2 Positionen bei künstlicher Intelligenz (nach John Searle):
 - **Schwache KI**
 - Es sollen mentale Zustände simuliert werden, ohne dass das System ein Bewusstsein besitzt und die Welt erlebt
 - **Starke KI**
 - Das System kann bei geeigneter Programmierung ein Bewusstsein entwickeln, das dem intelligenter Wesen (wie Menschen) entspricht
- Welche Position wahr und/oder erreichbar ist, ist nur schwer erfassbar!

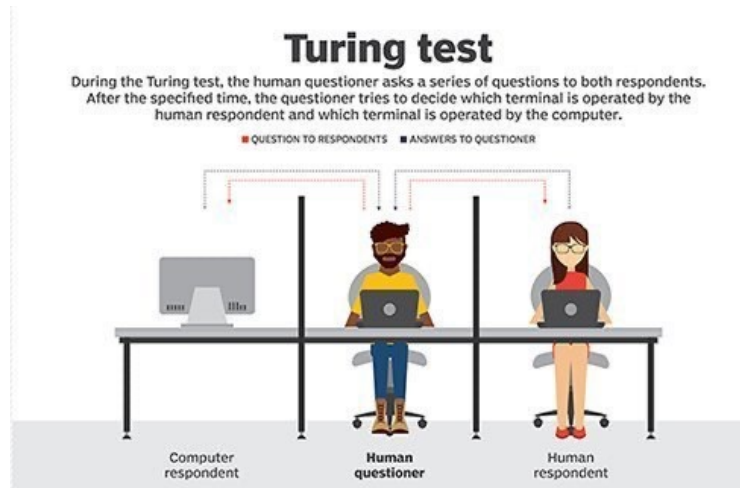


- **Turing-Test**

- Die Frage, ob Computer denken können, kann mit einem Test beantwortet werden (Alan Turing)
 - C stellt Fragen an A und B um zu entscheiden, wer von beiden ein Computer ist
 - A und B versuchen C zu überzeugen, dass sie Menschen sind
 - Kann C sich nicht entscheiden oder entscheidet sich für A, hat A den Turing-Test bestanden
- Besteht der Computer den Turing-Test, gibt es nach Alan Turing keinen Grund, ihm Intelligenz abzusprechen



[Wikimedia \(2017\)](#)



[Techtarget \(2021\)](#)

- **ELIZA Chatbot**

- An dem Test wurde viel Kritik geübt
- Joseph Weizenbaum hat ELIZA entwickelt, um seine Kritik zu zeigen
- ELIZA formuliert die Aussagen des menschlichen Gesprächspartners um und simuliert so eine Reaktion auf die Aussage
- Dabei entstehen Dialoge, die an gesprächstherapeutische Ansätze erinnern und ELIZA damit nicht als Computer identifizierbar machen
- ELIZA kann auch Schlüsselwörter identifizieren (wie z.B. „Vater/Mutter“) und reagieren, sodass der Eindruck von Sprachverstehen entsteht

- ELIZA Chatbot

```
Welcome to

EEEEEE LL      IIII ZZZZZZZ AAAAA
EE      LL      II      ZZ  AA  AA
EEEEEE LL      II      ZZZ  AAAAAA
EE      LL      II      ZZ  AA  AA
EEEEEE LLLLLL IIII ZZZZZZZ AA  AA

Eliza is a mock Rogerian psychotherapist.
The original program was described by Joseph Weizenbaum in 1966.
This implementation by Norbert Landsteiner 2005.

ELIZA: Is something troubling you ?
YOU:   Men are all alike.
ELIZA: What is the connection, do you suppose ?
YOU:   They're always bugging us about something or other.
ELIZA: Can you think of a specific example ?
YOU:   Well, my boyfriend made me come here.
ELIZA: Is it important to you that your boyfriend made you come here ?
YOU:   He says I'm depressed much of the time.
ELIZA: I am sorry to hear that you are depressed.
YOU:   It's true. I am unhappy.
ELIZA: Can you explain what made you unhappy ?
YOU:   █
```

[Wikimedia \(2018\)](#)

- **ELIZA Chatbot**
 - Beispiel
 - Benutzer: „Ich habe ein Problem mit meinem Auto“
 - ELIZA: „Warum haben Sie ein Problem mit Ihrem Auto?“
 - Benutzer: „Ich habe ein Problem mit meinem Vater“
 - ELIZA: „Erzählen Sie mir mehr über Ihre Familie“
 - Nach Weizenbaum besitzt ELIZA keine Intelligenz, auch wenn Gesprächspartner den Chatbot für einen Menschen halten
 - Der Turing Test stellt damit kein valides Verfahren dar, um maschinelle Intelligenz zu ermitteln

- **Chinesisches Zimmer**

- Ein weiterer Kritiker des Turing Tests ist John Searle
- Seiner Auffassung nach funktioniert der menschliche Geist weder wie ein Computerprogramm, noch können Computerprogramme wie Menschen denken oder ein Bewusstsein haben
- Menschliches Denken besteht nach Searle hauptsächlich aus **semantischen** Inhalten
 - Computer können aber nur auf einer **syntaktischen** Ebene (Symbole miteinander verknüpfen) arbeiten
 - Um dieses Position zu verdeutlichen, hat er ein Gedankenexperiment entwickelt: Das chinesische Zimmer

- **Chinesisches Zimmer**

„Andrea ist im Besitz einer in ihrer Muttersprache verfassten Anleitung zur korrekten Beantwortung von Fragen in chinesischer Sprache. Sie steht in einem ansonsten leeren Zimmer vor einer Tür, unter der immer wieder Zettel durchgeschoben werden, auf denen Fragen in chinesischer Sprache stehen. Obwohl Andrea selbst kein Chinesisch spricht, kann sie die chinesische Fragen mit Hilfe der Anleitung korrekt beantworten. Personen außerhalb des Zimmers folgern aus den Ergebnissen, dass Andrea Chinesisch spricht, obwohl sie nur Zeichenreihen manipuliert, ohne diese zu verstehen.“

[Philoclopedia \(2019\)](#)

- Chinesisches Zimmer

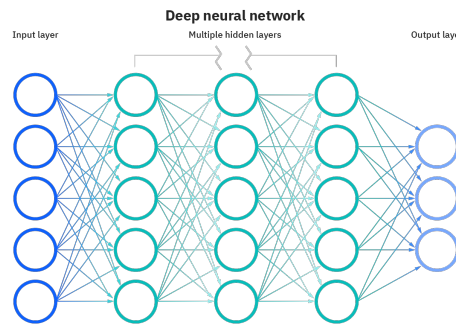


[Philoclopedia \(2019\)](#)

- **Chinesisches Zimmer**
 - Ein Computer kann also ein Programm ausführen (z.B. Übersetzung), muss aber die Bedeutung nicht verstehen
 - Eine Syntax zu befolgen ist nicht mit der Semantik verbunden
 - Computer, die Fragen korrekt beantworten können, müssen nicht auch ein Bewusstsein besitzen
 - Der Turing-Test ist somit **nicht ausreichend**, um starke künstliche Intelligenz zu beweisen

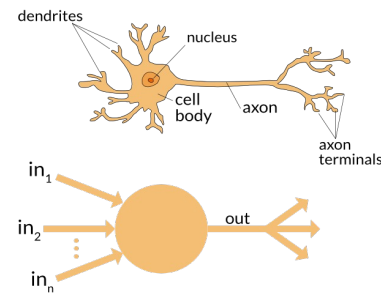
- Maschinelles Lernen wird im wesentlichen in 3 Arten geteilt:
 - Überwachtes Lernen
 - Unüberwachtes Lernen
 - Verstärktes Lernen
- Schwache KI bezeichnet KI ohne Bewusstsein, während bei starker KI von einem Bewusstsein ausgegangen wird
- Der Turing Test ist ein Versuch, um einzuschätzen, ob die KI ein Bewusstsein erlangt hat
- Mit dem ELIZA Chatbot und dem chinesischen Zimmer als Kritik wird der Turing Test aber als unzureichend erachtet

- Tiefes Lernen (Deep Learning)
 - Teilgebiet des maschinellen Lernens
 - Tief: es gibt mehrere Schichten
 - Anwendung meist mit künstlichen neuronalen Netzen (KNN)



IBM (2020)

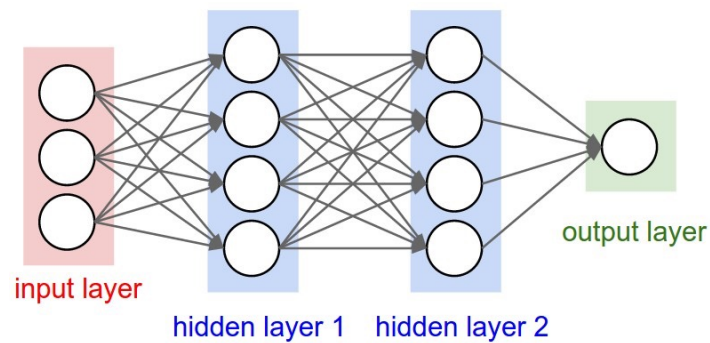
- **Ursprung:** Perzeptron
 - Ein Neuron wird als Verarbeitungseinheit angesehen, das mehrere Eingänge und einen Ausgang hat
 - Ein einfaches Perzeptron besteht aus einem einzelnen Neuron, das als Boolesche Variable konzipiert ist (wahr/falsch)
 - Das Neuron feuert („wahr“ wird ausgegeben), wenn die Summe der Eingangssignale einen Schwellenwert übersteigt
 - Die Funktionsweise ist damit analog zu einem biologischen Neuron (Aktionspotenzial: Wenn die Schwellenspannung überschritten wird, depolarisiert das Neuron)
 - Ein biologisches Neuron wird somit als Modell für den Aufbau eines künstlichen neuronalen Netzes verwendet



AppliedGo (2016)

- Die Bausteine des neuronalen Netztes bestehen aus vernetzten Verarbeitungseinheiten
 - Analog zu Neuronen und ihren synaptischen Verbindungen im Gehirn
- Dabei werden 3 Arten von Neuronen (Units) unterschieden:
 - Input Units
 - Hidden Units
 - Output Units
 - Sie sind in mehreren Schichten angeordnet
- Die Verbindungen haben spezifische Stärken und können inhibitorisch oder exzitatorisch wirken

- **Ziel:** Auf eine bestimmte sensorische Eingabe (Input Units) soll das neuronale Netz lernen, mit einer bestimmten Antwort (Output Units) zu reagieren
- Die Eingabe erfolgt über die Input Units, wird an die Hidden Units weitergeleitet und an die Output Units übergeben, über die die Antwort erfolgt



[StackOverflow \(2017\)](#)

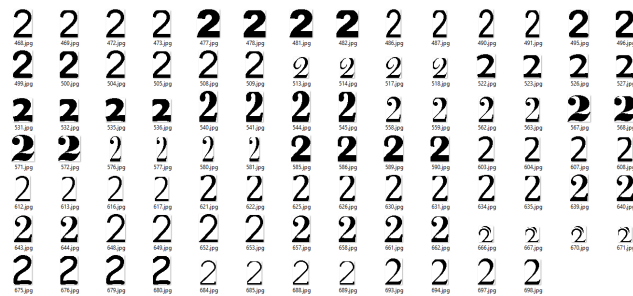
▪ Anwendungsbeispiele:

▪ Klassifikation von Rechtecken

- Immer, wenn eine Form der Art  vorliegt, antworte mir „ja“, andernfalls mit „nein“

▪ Erkennen von Zeichen und Ziffern

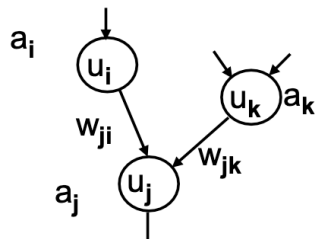
- Immer, wenn eine dieser oder ähnliche Strukturen vorliegen, identifiziere sie als Zahl 2



[StackOverflow \(2016\)](#)

- Funktionsweise des neuronalen Netzes
 - Ein neuronales Netz erzeugt eine gewünschte Antwort zu einem gegebenen Stimulus
 - Der Stimulus kommt aus der Umgebung und wirkt auf die Input Units
 - Die Input Units generieren ein spezifisches Erregungsmuster und leiten es über die Verbunden wieder
 - Die Hidden Units (Zwischenschichten) realisieren über ihre Aktivierungsmuster spezifische Verarbeitungsschritte
 - Sie stellen sicher, dass das Eingangssignal zu dem gewünschten Ausgangssignal führt
 - Die Antwort wird repräsentiert als Erregungsmuster der Output Units

- Struktur des neuronalen Netzes
 - Unterscheidung zwischen Graph und Matrixschreibweise



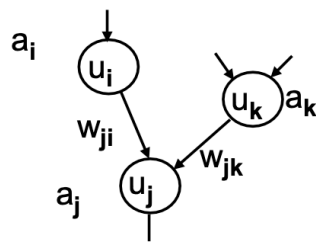
	u_i	u_j	u_k	...
u_i	0	w_{ji}	0	
u_j		0	w_{jk}	
u_k			0	
...				

- Struktur des neuronalen Netzes

u_i : Unit i

a_i : Aktivierung der Unit i

w_{ij} : Gewicht der Verbindung zwischen Unit i (sendend) und Unit j (empfangend)



	u_i	u_j	u_k	...
u_i	0	w_{ji}	0	
u_j		0	w_{jk}	
u_k			0	
...				

Der erste Index im Gewicht bezeichnet die empfangende Unit, der zweite Index die sendende Unit

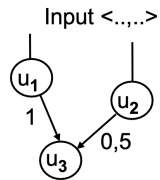
Verarbeitungsannahmen

- Die Neuronen jeder Schicht zeigen ein Verhaltensmuster aus 4 Schritten
 1. **Aufnahme der Inputs** aller sendenden Neurone, die mit dem Zielneuron verbunden sind
 2. Berechnung des **Gesamteingangssignals** aus den einzelnen Eingangssignalen (Netinput)
 3. Berechnung der eigenen **aktuellen Aktivierung** (Aktivationslevel)
 4. Berechnung des aus der Aktivierung resultierenden **Ausgangssignals**
- Die letzten 3 Schritte werden durch 3 Funktionen umgesetzt, die die numerische Verarbeitung einer Unit repräsentieren

- Drei Funktionen
 - **Net Input Function (net)**
 - Funktion der Outputs (o) der vorgeschalteten Units sowie der Gewichte (w) der Verbindungen
 - Meist realisiert als gewichtete Summe:
$$net_j = \sum_{i=1}^I w_{ji} \cdot o_i$$
 - **Activation Function (a)**
 - F bestimmt für jede Unit die Aktivierung a:
$$a_j = f(net_j)$$
 - **Output Function (o)**
 - Für jede Unit existiert eine Aufgabe, die aus der Aktivierung folgt: $o_j = f(a_j)$
 - Häufig wird hierfür die Identitätsfunktion verwendet: $o_j = a_j$

- Drei Funktionen – Beispiel

- Beispiel:

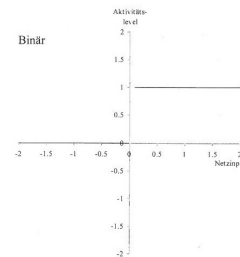
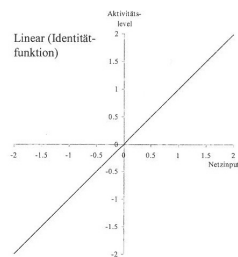


- Net Input Funktion:

$$net_j = \sum_{i=1}^I w_{ji} \cdot o_i$$

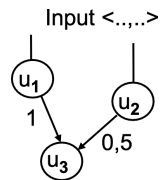
	u_1	u_2	u_3
Output	$o_1 = -1$	$o_2 = 4$	
Net Input			$net_3 = 1 \cdot -1 + 0.5 \cdot 4 = 1$

- Drei Funktionen – Beispiel
 - Aktivierungsfunktion: $a_j = f(\text{net}_j)$
 - Es können verschiedene Aktivierungsfunktionen zum Einsatz kommen
 - 3 Beispiele:
 - Linearer Zusammenhang zwischen Aktivierung und Net Input: $a_i = m + n \cdot \text{net}_i$
 - Spezialfall Identitätsfunktion: $a_i = \text{net}_i$, mit $m = 0$ und $n = 1$
 - Binär : $a_i = 0$ für net_i kleiner gleich 0, $a_i = 1$ für net_i größer 0



- Drei Funktionen – Beispiel

- Beispiel:



- Binäre Aktivierungsfunktion: $a_j = \begin{cases} 0 & \text{für net}_j \text{ kleiner gleich } 0 \\ 1 & \text{für net}_j \text{ größer } 0 \end{cases}$
 - Outputfunktion: $o_j = a_j$

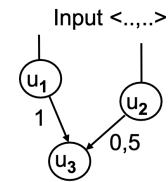
- Drei Funktionen – Beispiel
- Mit den gegebenen Funktionen, Aktivierungen und Gewichten kann das Netz nun simuliert werden

- Net Input Funktion: $net_j = \sum_{i=1}^I w_{ji} \cdot o_i$

- Binäre Aktivierungsfunktion: $a_j = \begin{cases} 0 & \text{für } net_j \text{ kleiner gleich } 0 \\ 1 & \text{für } net_j \text{ größer } 0 \end{cases}$

- Outputfunktion: $o_j = a_j$

- **Schritt 1:**



	u_1	u_2	u_3
Net Input	$net_1 = 4$	$net_2 = 0.5$	
Aktivierung			
Output			

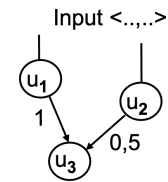
- Drei Funktionen – Beispiel
- Mit den gegebenen Funktionen, Aktivierungen und Gewichten kann das Netz nun simuliert werden

- Net Input Funktion: $net_j = \sum_{i=1}^I w_{ji} \cdot o_i$

- Binäre Aktivierungsfunktion: $a_j = \begin{cases} 0 & \text{für } net_j \text{ kleiner gleich } 0 \\ 1 & \text{für } net_j \text{ größer } 0 \end{cases}$

- Outputfunktion: $o_j = a_j$

- **Schritt 2:**



	u_1	u_2	u_3
Net Input	$net_1 = 4$	$net_2 = 0.5$	
Aktivierung	$a_1 = 1$	$a_2 = 1$	
Output			

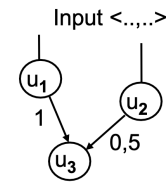
- Drei Funktionen – Beispiel
- Mit den gegebenen Funktionen, Aktivierungen und Gewichten kann das Netz nun simuliert werden

- Net Input Funktion: $net_j = \sum_{i=1}^I w_{ji} \cdot o_i$

- Binäre Aktivierungsfunktion: $a_j = \begin{cases} 0 & \text{für } net_j \text{ kleiner gleich } 0 \\ 1 & \text{für } net_j \text{ größer } 0 \end{cases}$

- Outputfunktion: $o_j = a_j$

- **Schritt 3:**



	u_1	u_2	u_3
Net Input	$net_1 = 4$	$net_2 = 0.5$	
Aktivierung	$a_1 = 1$	$a_2 = 1$	
Output	$o_1 = 1$	$o_2 = 1$	

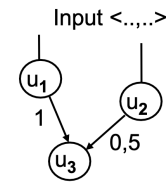
- Drei Funktionen – Beispiel
- Mit den gegebenen Funktionen, Aktivierungen und Gewichten kann das Netz nun simuliert werden

- Net Input Funktion: $net_j = \sum_{i=1}^I w_{ji} \cdot o_i$

- Binäre Aktivierungsfunktion: $a_j = \begin{cases} 0 & \text{für } net_j \text{ kleiner gleich } 0 \\ 1 & \text{für } net_j \text{ größer } 0 \end{cases}$

- Outputfunktion: $o_j = a_j$

- **Schritt 4:**



	u_1	u_2	u_3
Net Input	$net_1 = 4$	$net_2 = 0.5$	$net_3 = 1 * 1 + 0.5 * 1 = 1.5$
Aktivierung	$a_1 = 1$	$a_2 = 1$	
Output	$o_1 = 1$	$o_2 = 1$	

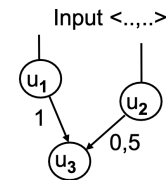
- Drei Funktionen – Beispiel
- Mit den gegebenen Funktionen, Aktivierungen und Gewichten kann das Netz nun simuliert werden

- Net Input Funktion: $net_j = \sum_{i=1}^I w_{ji} \cdot o_i$

- Binäre Aktivierungsfunktion: $a_j = \begin{cases} 0 & \text{für } net_j \text{ kleiner gleich } 0 \\ 1 & \text{für } net_j \text{ größer } 0 \end{cases}$

- Outputfunktion: $o_j = a_j$

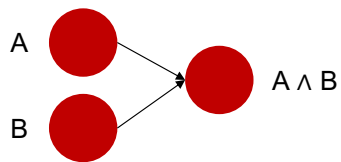
- **Schritt 5:**



	u_1	u_2	u_3
Net Input	$net_1 = 4$	$net_2 = 0.5$	$net_3 = 1 * 1 + 0.5 * 1 = 1.5$
Aktivierung	$a_1 = 1$	$a_2 = 1$	$a_3 = 1$
Output	$o_1 = 1$	$o_2 = 1$	$o_3 = 1$

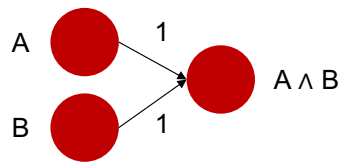
- Verknüpfung mit Aussagenlogik
 - Aussagen können in neuronalen Netzen dargestellt werden
 - Beispiel: Darstellung der **Und-Relation** der Aussagenlogik

A	B	$A \wedge B$
+	+	+
+	-	-
-	+	-
-	-	-

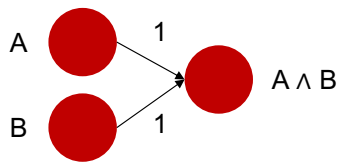


Nur wenn sowohl A als auch B wahr sind, gib aus, dass die Konjunktion „A und B“ wahr ist

- Verknüpfung mit Aussagenlogik
 - Übung:
 - Sowohl A, als auch B haben die Gewichtung 1
 - Die Outputfunktion entspricht $o_j = a_j$
 - Wie sieht die Aktivierungsfunktion von $A \wedge B$ aus?

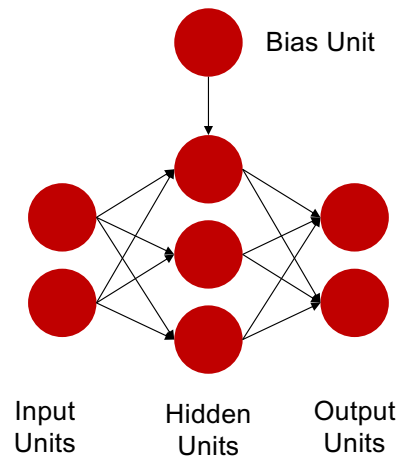


- Verknüpfung mit Aussagenlogik
 - Übung:
 - Sowohl A, als auch B haben die Gewichtung 1
 - Die Outputfunktion entspricht $o_j = a_j$
 - Wie sieht die Aktivierungsfunktion von $A \wedge B$ aus?
 - Lösung: $a_{A \wedge B} = \begin{cases} 0 & \text{für } \text{net}_{A \wedge B} \text{ kleiner gleich } 1 \\ 1 & \text{für } \text{net}_{A \wedge B} \text{ größer } 1 \end{cases}$



- Bias Units
 - Neben den drei Unit Arten (Input Unit, Hidden Unit, Output Unit) gibt es noch Bias Units
 - Bias Units erhalten keinen Input, sondern haben eine konstante Aktivierung von 1
 - Damit stellen sie eine Voraktivierung ihres Zielneurons dar
 - Ist das Gewicht ihrer Verbindung positiv, bekommt das Zielneuron auch dann eine Aktivierung, wenn kein anderer Input von anderen Neuronen vorliegt
 - Ist das Gewicht negativ, dämpft die Bias Unit die Aktivierung des Ziels
 - In Kombination mit einer Aktivierungsfunktion, die eine Schwelle erzeugen soll, sind Bias Units besonders nützlich
 - Die Schwelle ist nicht festgelegt, sondern durch das veränderbare Gewicht der Verbindung variabel

- Bias Units



- Lernregeln
 - Lernregeln legen fest, wie die Gewichte eines neuronalen Netzes verändert werden
 - Zwei bekannte Verfahren sind die Hebb-Regel und die Delta-Regel
 - **Hebb-Regel:**
 - Hat den Anspruch hoher physiologischer Plausibilität. „What fires together wires together“
 - $\Delta w_{ij} = \epsilon * a_i * a_j$
 - Δw_{ij} : Veränderung des Gewichts zwischen U_i und U_j
 - ϵ : Vorher festgelegter Lernparameter (meist <1)
 - a_i : Aktivierung (bzw. Output) von U_i (empfangende Unit)
 - a_j : Aktivierung (bzw. Output) von U_j (sendende Unit)

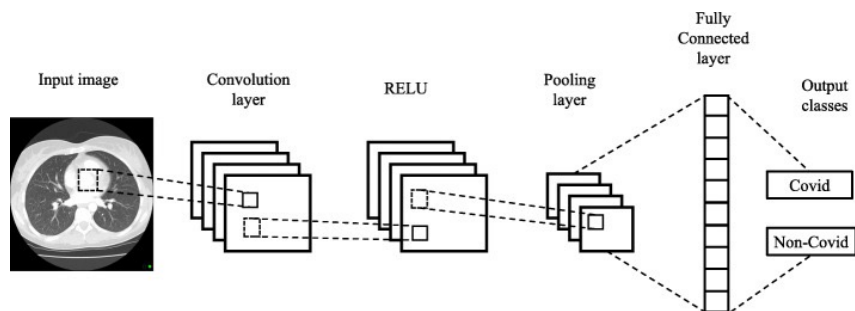
- Lernregeln
 - **Hebb-Regel:** $\Delta w_{ij} = \varepsilon * a_i * a_j$
 - Beispiel Heb-Regel:
 - Das ursprüngliche Gewicht zwischen U_i und U_j beträgt zum Beispiel +3, die Aktivierung der Units jeweils +1 und der Lernparameter +0.5
 - Damit ist $\Delta w_{ij} = 0.5 * 1 * 1 = 0.5$ und das neue Gewicht $w_{ij} = 3 + 0.5 = 3.5$
 - Die Hebb-Regel **erhöht** hier also in Abhängigkeit der gemeinsamen Aktivierung zweier Units das Gewicht ihrer Verbindung
 - Sie kann beim **überwachten** und auch beim **unüberwachten** Lernen eingesetzt werden

- Lernregeln
 - **Delta-Regel:**
 - Kommt beim überwachten Lernen zum Einsatz und vergleicht den gewünschten Output und dem (in dem aktuellen Durchgang) tatsächlichen Output:
 - $\delta = a_i (\text{gewünscht}) - a_i (\text{beobachtet})$
 - Der Vergleich kann drei Ergebnisse haben:
 - Die beobachtete Aktivität ist zu niedrig, also $\delta > 0$ und das Gewicht muss erhöht werden
 - Die beobachtete Aktivität ist zu hoch, also $\delta < 0$ und das Gewicht muss verringert werden
 - Beobachtete und gewünschte Aktivität sind gleich, also $\delta = 0$ und das Gewicht bleibt unverändert
 - $\Delta w_{ij} = \epsilon * \delta_i * a_j$
 - Δw_{ij} : Veränderung des Gewichts zwischen U_i und U_j
 - ϵ : Vorher festgelegter Lernparameter (meist < 1)
 - δ : Deltawert (Differenz von gewünschtem und beobachtetem Output)
 - a_i : Aktivierung (bzw. Output) von U_i (sendende Unit)

- Lernregeln
 - **Delta-Regel:** $\Delta w_{ij} = \varepsilon * \delta_i * a_j$
 - Beispiel Delta-Regel:
 - Die Aktivität einer Output Unit i soll 5 betragen, die berechnete Aktivität beträgt jedoch nur 1
 - Daraus ergibt sich ein Delta von 4, sodass das Gewicht erhöht werden muss
 - Bei einem Lernparameter von 1 und einer Aktivität der sendenden Unit j von 0.5 beträgt die Gewichtsveränderung $1 * 4 * 0.5 = 2$
 - Wenn das bestehende Gewicht den Wert 1 hat, so hat das neue Gewicht den Wert $1 + 2 = 3$
 - Weitere bekannte Lernregeln sind z.B. das Gradientenabstiegsverfahren und Backpropagation

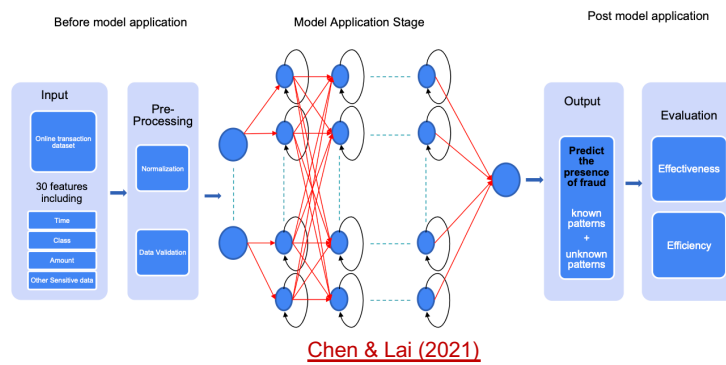
- Grenzen von neuronalen Netzen
 - Trainingsdaten
 - Um neuronale Netze zu trainieren müssen umfassende Trainingsdaten vorhanden sein
 - In den Daten können Biases reproduziert sein
 - In den Daten können systematische Merkmale vorhanden sein, auf die das neuronale Netz trainiert wird (z.B. blauer Himmel bei der Klassifikation von Häusern)
 - Hoher Rechenaufwand
 - Das Training der Netze dauert einige Zeit
 - Hoher Geld- oder Zeitaufwand
 - Mit entsprechenden Algorithmen kann der Aufwand verringert werden: Stochastisches Gradientenverfahren, Mini-Batch-Gradientenverfahren, etc.
 - Mangelnde Interpretierbarkeit
 - Neuronale Netze haben viele Neurone, denen ein Zahlenwert zugeordnet ist
 - Damit stellen sie eine Art Blackbox dar, die kaum interpretierbar ist

- Anwendungsszenarien von neuronalen Netzen
 - Diagnose von COVID-19
 - Training eines spezifischen neuronalen Netzwerks (CNN, Convolution Neural Network)
 - Input: Röntgen- und CT-Aufnahmen von COVID-19-Patient*innen
 - Accuracy: 93%



[Kugunavar & Prabhakar \(2021\)](#)

- Anwendungsszenarien von neuronalen Netzen
 - Erkennung von Kreditkartenbetrug
 - Training eines spezifischen neuronalen Netzwerks (DCNN, Deep Convolution Neural Network)
 - Input: Datenset von Onlinetransaktionen mit 30 Variablen
 - Accuracy: 99%



- Neuronale Netze bestehen aus 3 (4) Arten von Neuronen / Units:
 - Input Units
 - Hidden Units
 - Output Units
 - (Bias Units)
- Jede Unit bekommt einen Input, hat eine Aktivierungsfunktion und gibt einen entsprechenden Output aus
- Um die Netze zu trainieren gibt es mehrere Regeln, wobei die Hebb-Regel und die Delta-Regel zwei bekannte (und verständliche) Regeln sind
- Neuronale Netze haben vor allem Grenzen / Einschränkungen im Bezug auf ihre Trainingsdaten, auf den Rechenaufwand und sie können nur schwer interpretiert werden